Projeto: Arena de Combate RPG

Programação Orientada a Objetos com PHP

Contexto do Projeto

Você foi contratado para desenvolver o protótipo de um **jogo de combate por turnos** que roda inteiramente no terminal. O jogo simula uma arena onde dois combatentes se enfrentam em um duelo 1 contra 1. Cada jogador controla o seu respectivo personagem **no mesmo computador**, revezando as ações a cada turno — semelhante a jogos de tabuleiro clássicos onde os participantes se alternam na mesma tela.

O objetivo é colocar em prática os pilares da Programação Orientada a Objetos: **encapsulamento**, **herança**, **polimorfismo** e **abstração**. Além disso, você exercitará o uso de **interfaces**, **classes abstratas**, **tratamento de exceções** e a organização de código em múltiplos arquivos.

Regras do Jogo

Estrutura da Partida

O jogo é disputado entre exatamente **dois jogadores humanos**. Antes do combate começar, cada jogador deve escolher um tipo de personagem para usar na partida. Existem pelo menos dois tipos distintos de personagem disponíveis, cada um com características e habilidades únicas que influenciam diretamente na estratégia de combate.

Após ambos os jogadores escolherem seus personagens, o combate se inicia. Os jogadores se revezam a cada turno: o **Jogador 1** escolhe sua ação, ela é executada imediatamente, e então é a vez do **Jogador 2** fazer o mesmo. Esse ciclo se repete até que um dos personagens seja derrotado.

Atributos dos Personagens

Todo personagem, independente do tipo, possui os seguintes atributos numéricos:

- **Pontos de vida:** representam a saúde do personagem. Quando chegam a zero (ou menos), o personagem é derrotado. Não podem ultrapassar o valor máximo inicial.
- **Poder de ataque:** determina o dano base que o personagem causa ao adversário quando realiza um ataque.
- **Defesa:** reduz o dano recebido de ataques do oponente. O cálculo exato fica a seu critério, mas a defesa deve sempre reduzir o dano — nunca aumentá-lo.
- **Energia (ou mana):** recurso consumido ao usar a habilidade especial do personagem. Começa em um valor definido e pode se regenerar parcialmente a cada turno. Sem energia suficiente, a habilidade especial não pode ser usada.

> **Dica: Balanceamento**
> Os valores iniciais dos atributos devem ser diferentes entre os tipos de personagens. Por exemplo, um tipo mais ofensivo pode ter mais ataque e menos vida, enquanto um tipo mais defensivo pode ter mais vida e mais defesa. Procure fazer com que nenhum tipo seja objetivamente superior ao outro — cada um deve ter vantagens e desvantagens.

Ações Disponíveis por Turno

A cada turno, o jogador da vez deve escolher **exatamente uma** das seguintes ações:

1. Atacar

O personagem realiza um ataque direto contra o oponente. O dano causado deve considerar o poder de ataque de quem ataca e a defesa de quem recebe. O dano final **nunca pode ser negativo** (ou seja, se a defesa for maior que o ataque, o dano mínimo é zero). O dano é subtraído dos pontos de vida do oponente.

2. Defender

O personagem assume uma postura defensiva, **aumentando temporariamente sua defesa** até o próximo turno. Esse bônus de defesa deve ser removido automaticamente quando o turno do jogador começar novamente. Defender é uma escolha estratégica para turnos em que o jogador antecipa um golpe forte do oponente.

3. Usar Habilidade Especial

Cada tipo de personagem possui **uma habilidade especial única** que o diferencia dos demais. A habilidade consome energia e produz um efeito mais poderoso ou diferente de um ataque comum. Exemplos de efeitos possíveis (você não precisa usar estes; invente os seus):

- Causar dano elevado a custo de vida própria
- Recuperar parte dos pontos de vida
- Aplicar um efeito negativo no oponente (veneno, redução de ataque, etc.)
- Realizar um contra-ataque automático no próximo turno

Se o jogador tentar usar a habilidade especial sem energia suficiente, o sistema deve **impedir a ação e exibir uma mensagem clara**, permitindo que o jogador escolha outra ação no mesmo turno.

> **Atenção**
> As ações devem ser tratadas de forma polimórfica. Ou seja, o loop principal do jogo não deve precisar saber qual tipo específico de personagem está jogando para executar uma ação — ele apenas chama o método correspondente, e cada tipo de personagem responde de acordo com sua implementação.

Condição de Vitória

A partida termina imediatamente quando os pontos de vida de um dos personagens chegam a **zero ou menos**. O outro jogador é declarado vencedor. Ao final, o jogo deve exibir um resumo contendo: quem venceu, quantos turnos a partida durou e os pontos de vida restantes do vencedor.

Interface no Terminal

Toda a interação acontece via terminal usando `readline()` ou `fgets(STDIN)`. O jogo deve apresentar uma experiência clara e organizada:

Tela de Seleção de Personagem

Antes do combate, cada jogador vê um menu listando os tipos de personagem disponíveis, com uma breve descrição de cada um (por exemplo: nome do tipo e um resumo das suas características). O jogador digita o número correspondente a sua escolha. Se digitar algo inválido, o sistema deve solicitar novamente.

Tela de Combate (a cada turno)

A cada turno, o terminal deve exibir claramente:

- De quem é a vez (Jogador 1 ou Jogador 2)
- O nome e tipo de ambos os personagens
- Os pontos de vida atuais de ambos (e, opcionalmente, uma barra visual)
- A energia atual do personagem da vez
- As ações disponíveis numeradas

Após o jogador escolher a ação, o resultado deve ser exibido de forma descritiva. Por exemplo: *"Guerreiro atacou Mago causando 15 de dano! Mago agora tem 45 HP."*

> **Sugestão: Limpeza de Tela**
> Para melhorar a experiência visual, você pode limpar o terminal entre turnos utilizando a função `system('clear')`.

Requisitos Técnicos Obrigatórios

O projeto deve demonstrar os conceitos fundamentais de POO. Abaixo estão os requisitos que o seu código **deve** atender:

1. **Classe abstrata:** deve existir pelo menos uma classe abstrata que sirva de base para os tipos de personagem. Ela deve definir os atributos comuns e os métodos que todo personagem precisa ter, mas ao menos um desses métodos deve ser abstrato — ou seja, deve obrigar as subclasses a fornecerem suas próprias implementações.
2. **Herança:** os tipos de personagem devem herdar da classe abstrata, reaproveitando atributos e comportamentos comuns e implementando suas particularidades (como a habilidade especial).
3. **Encapsulamento:** os atributos dos personagens devem ser protegidos ou privados. O acesso e a modificação devem ocorrer exclusivamente por meio de métodos. Nenhum atributo pode ser público.
4. **Polimorfismo:** o loop de combate deve ser capaz de trabalhar com qualquer tipo de personagem sem verificar qual tipo específico é. A chamada dos métodos (atacar, defender, usar habilidade) deve funcionar de forma polimórfica.
5. **Tratamento de exceções:** o jogo deve utilizar exceções personalizadas (pelo menos uma classe de exceção própria) para tratar situações como: energia insuficiente para habilidade especial, entrada inválida do jogador, ou qualquer outra situação de erro.
6. **Organização em arquivos:** cada classe deve estar em seu próprio arquivo. O ponto de entrada do jogo (o arquivo que você executa) deve incluir os demais arquivos usando `require_once` ou um autoloader. Não coloque tudo em um único arquivo.
7. **Constantes de classe:** utilize pelo menos uma constante de classe (com `const`) onde fizer sentido. Por exemplo: valor máximo de energia, dano mínimo, bônus de defesa, etc.

Etapas Sugeridas de Desenvolvimento

Você não precisa seguir esta ordem, mas ela pode ajudar a organizar seu desenvolvimento de forma progressiva:

1. **Estrutura as classes base:** comece criando a classe abstrata com os atributos comuns e os métodos (abstratos e concretos). Crie a(s) interface(s).
2. **Implemente os tipos de personagem:** crie pelo menos dois tipos concretos, cada um com seus valores de atributos e sua habilidade especial.
3. **Implemente as ações:** faça o ataque, a defesa e a habilidade especial funcionarem corretamente, respeitando as regras de cálculo de dano, bônus temporário de defesa e consumo de energia.
4. **Monte o loop de combate:** crie o fluxo principal do jogo: alternância de turnos, leitura de entrada, execução de ação e verificação de fim de jogo.
5. **Adicione a seleção de personagem:** crie o menu inicial onde cada jogador escolhe seu tipo.
6. **Trate os erros:** implemente as exceções personalizadas e garanta que entradas inválidas e ações impossíveis sejam tratadas adequadamente.
7. **Refine a interface:** melhore as mensagens exibidas no terminal, adicione feedback visual e o resumo final da partida.

Desafios Extras (Opcionais)

Se você terminou o projeto base e quer ir além, tente implementar um ou mais dos seguintes desafios:

- **Terceiro tipo de personagem:** adicione um novo tipo com atributos e habilidade especial completamente diferentes dos dois originais.
- **Sistema de efeitos ao longo do tempo:** implemente efeitos que duram mais de um turno, como veneno (dano a cada turno), queimadura, enfraquecimento, etc. Esses efeitos devem ser aplicados automaticamente e ter duração limitada.
- **Log de combate:** armazene todas as ações e resultados da partida em um array e, ao final, exiba o histórico completo da batalha, separado por turnos.
- **Barra de vida visual:** exiba a vida dos personagens com uma barra gráfica no terminal usando caracteres como `[##########----------]`.

Entrega

- Link do repositório público no GitHub
- O jogo deve ser executável com o comando `php nome-do-arquivo-principal.php` no terminal.
- O código deve rodar sem erros no `PHP 8.1` ou superior.
- Inclua um breve arquivo `README.md` explicando: como executar o jogo, quais tipos de personagem existem e o que cada habilidade especial faz.

Bom desenvolvimento e boa batalha!